FCR: FLOW CHART RECOGNITION NETWORK FOR PROGRAM SYNTHESIS

Anonymous authors
Paper under double-blind review

ABSTRACT

Program synthesis is the task to automatically generate programs based on user specification. In this paper, we present a framework that synthesizes programs from images of flow chart that serve as accurate and intuitive specifications. In order doing so, we propose a deep neural network called FCR that recognizes graph structure from its image. FCR is trained end-to-end, which can predict edges and nodes of the flow chart simultaneously. Experiments show that the accuracy to synthesize a program is 66.4%, and the accuracy to recognize edges and nodes are 94.1% and 67.9%, respectively. On average, it takes about 60 milliseconds to synthesize a program.

1 INTRODUCTION

Program synthesis enables people to program computers without training in coding. It has been used in many domains such as data wrangling, graphs, and code repair Gulwani (2010). A good example is FlashFill Gulwani (2011), which allows spreadsheet users to provide a few examples and generates a program that conforms to the examples.

To synthesize a program, specification must be provided. Specification in formal language can accurately represent the user intent and is used in deductive program synthesis Manna & Waldinger (1980). However, very few have the knowledge of formal language, so it cannot benefit most end users. Under-specification is used in programming by example Gulwani (2011); Feng et al. (2017), and programming by demonstration Lau & Weld (1998). Under-specification does not require language knowledge and is accessible by most end users. However, because there may be more than one program that satisfy the specification, how to choose the correct program that captures the user intent is still an open problem.

In this paper, we propose a new technique called FCR (Flow Chart Recognition Convolutional Neural Network) that synthesizes programs from images of flow chart. Flow charts are diagrams that represent the workflow of programs. They are widely used in textbooks to teach coding and illustrate programs. Because flow charts are intuitive and accurate, they allows users without programming knowledge to accurately specify the programs. Figure 1 shows the flow chart of the `absolute` function.

Due to recent progress in deep learning, CNN has beome a tool that is able to recognize complicated information from image. Therefore, we can predict edges and nodes from the image of flow chart with the help of deep CNN. Moreover, by employing neural networks, our method does not suffer from the combinatorial explosion problem that plagues traditional program synthesis methods.

Figure 1 givens an overview of our approach. First, an image of flow chart is resized to a fixed size and fed to FCR. Then, FCR generates the graph information including an adjacent matrix for the edges and a list of strings for the nodes. Finally, we compile the graph information to source code.

Experiments on our synthetic test dataset show that the accuracy to synthesize a program is 66.4%, and the accuracy to predict edges and nodes are 94.1% and 67.9%, respectively. Experiments on another dataset, which is manually converted from a textbook, show that the accuracy to synthesize a program is 63.6%, and the accuracy to predict edge and nodes are 72.7% and 81.8%, respectively. The average time to synthesize a program is about 60 milliseconds. Our evaluation also show that it is feasible to share the convolutional vector between edge and node networks.

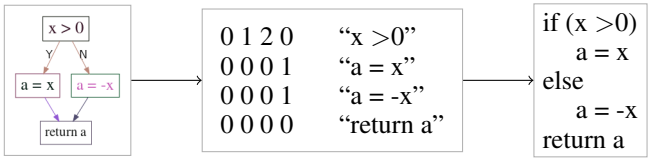


Figure 1: Overview of FCR. The input is the flow chart of `abs` function. The middle is the adjacent matrix and text of each node generated by FCR. The output is the synthesized source code.

Our contributions are:

- We propose a deep neural network that parses the graph information, including edge and node information, from the image of a flow chart.
- We propose flow charts as accurate and intuitive specification for program synthesis.
- We implemented a prototype neural network and conducted empirical study of FCR.

2 Related Work

Traditional methodology of program synthesis is to construct a program space and design search algorithms to find a solution that satisfies the specification. The program space usually grows exponentially with the size of the target program. Different methods are proposed to speedup the search. For example, Flashfill, Gulwani (2011), synthesizes string-transforming programs given input-output examples. It uses dynamic programming to speedup the search. Morpheus, Wang et al. (2017), enumerates nested queries and prunes by grouping programs with same input-output pairs. Sketch, Solar-Lezama & Bodik (2008), and Sqlsol, Cheng (2019), encode the synthesis problem into logic constraints and delegate the searching algorithm to modern SMT solvers. This approach can boost the performance because modern SMT solvers are implemented for efficiency. Though different methods are proposed to speedup the synthesis process, the underlying complexity is unchanged and scalability is still an issue when it comes to large programs. Another issue is how to capture users intent. Typically, synthesis algorithms terminate when the first solution is found or the best-ranking solution is found. They may ask a user to provide more examples. However, there is no guarantee that the solution precisely captures the users intent.

Recently researchers propose to use machine learning to speed up the search for synthesized program. Morpheus Feng et al. (2017) uses statistics to rank R program sketches, and uses the rank to guide the search. DeepCoder Balog et al. (2016) augments beam search with deep learning recommendation, and the speedup is significant. However, the underlying complexity is still unchanged.

Researchers proposed natural language based program synthesis techniques. SQLizer, Yaghmazadeh et al. (2017), synthesizes SQL queries from natural language. Locascio et al, Locascio et al. (2016), synthesizes regular expressions from natural language. FCR differs from these approaches in that our input is different. Flow charts can accurately specify the users intent, while natural language is ambiguous.

Faster RCNN, Ren et al. (2015), and LPRNet, Zherzdev & Gruzdev (2018), are closely related to our work. Two subnets of FCR are built following their ideas. Faster RCNN is a deep neural network for object detection. It slides a small window on a convolutional feature and generates box proposals relative to anchors at each position. The box proposal is used to crop the image for a classifier to detect the class of the object in it. Because of the shape and positioning of nodes in a flow chart are different from those in general object-detection jobs, FCR chooses different anchors and methods to select the proposals. LPRNet is a license plate recognizing deep convolutional neural network. It reads an image of a license plate and generates a sequence that preserves the spatial order of the characters. A feature of LPRNet is that it has only one deep convolutional network, while other work has both a CNN for feature extraction and a recurrent neuralnetwork for prediction. This feature helps to limit the number of subnets in FCR, since it already has four subnets. Instead of taking a license plate as input, FCR takes as input a crop of image whose boundary is predicted by another network.

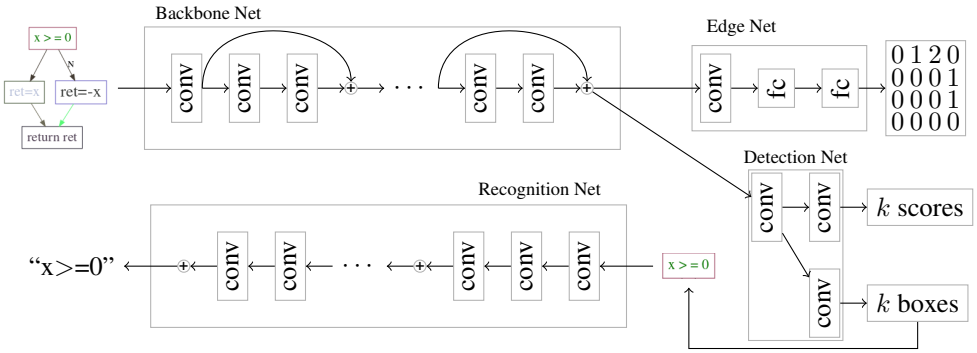


Figure 2: Architecture of FCR

3 NETWORK

In this section, we first describe the network and its subnets. Then, we describe the loss function and training details.

The input of FCR is the image of a flow chart. A flow chart is a graph diagram that represents the work flow of a program, Shelly & Vermaat (2011). The input image is resized to a fixed size, 400×200, before being fed to the neural network. If both height and width are less then the fixed size, we pad the image by zeroes. Otherwise, we interpolate the image to the fixed size. The output of FCR is graph representation of the flow chart, including an adjacent matrix for the edge representation and a list of text for content in nodes.

Figure 2 is the overview of FCR. FCR has four subnets: backbone network, edge network, node detection network, and node recognition network. The backbone network takes as input the raw image of the flow chart and outputs a feature vector. The feature vector is fed to both the edge to produce the adjacent matrix of the flow chart, and the node detection network to generate the bounding boxes of each node, which is used to crop the node from the original image. Then, the crop of each node is fed to the node recognition network to generate the text in the node.

3.1 BACKBONE NETWORK

Table 1 shows the architecture of a basic block, which takes as input a feature vector of C_{in} channels and outputs a feature vector of C_{out} channels of the same height and width. Each convolutional layer in the basic block is followed by a batch normalization layer and a ReLU activation layer.

The backbone network is a deep CNN that takes as input an image of size $C_{in} \times H \times W$. The output is a rich feature vector which is later feed to the edge network and the node detection network. The backbone network is a sequence of four basic blocks. Each of the first three basic blocks is followed by a max-pooling layer. During training, a dropout layer (p=0.1) is added after each pooling layer.

Optionally, the backbone network can be made a residual learning network (ResNet), He et al. (2016), by performing a down-sampling on the input vector with 1×1 convolutional layer to C_{out} channels, and adding the result to the original output of the backbone network.

3.2 EDGE NETWORK

The edge network takes as input the feature vector generated by the backbone network and outputs an adjacent matrix, which is the edge representation of the flow chart. The element of the adjacent matrix at row i and column j has three possible values: 0, 1, 2, which encode no edge, normal edge or `YES` branch of a decision node, `NO` branch of a decision node from node i to node j, respectively. Because the number of nodes in the flow chart may vary, we pad the adjacent matrix to a fixed size PAD by zeroes. We set $PAD = 6$ in this work. We encode the three values in the adjacent matrix with one-hot-vector of length 3. The edge network outputs a vector of size $PAD \times PAD \times 3$.

Table 1: Basic block architecture.

Layer Type	Parameters
Input	$C_{in} \times H \times W$
Conv2D	$C_{out}/4, 3 \times 3$
Conv2D	$C_{out}/4, 3 \times 3$
Conv2D	$C_{out}/4, 3 \times 3$
Conv2D	$C_{out}, 3 \times 3$

Table 2: Backbone network architecture

Layer Type	Parameters
Input	C × H × W
Basic Block	15, 3 × 3
Max Pool	2 × 2
Basic Block	50, 3 × 3
Max Pool	2 × 2
Basic Block	200, 3 × 3
Basic Block	400, 3 × 3
Max Pool	2 × 2

The edge network first performs a 3×3 convolutional layer activated by a ReLU layer followed by a 3×3 max-pooling layer. Then, it performs two linear layers ($\times 400$ and $\times 108$) activated by Tanh layers. We observed the training converges significantly faster when using Tanh activation function in the linear layers than using other activation functions.

The loss function to train the edge network is the multi-class multi-classification hinge loss . Equation 1 is the formula of the loss function, where x is the input vector and y is the target class indices.

$$l_e(x, y) = \sum_{i,j} \frac{max(0, 1 - (x[y[j]] - x[i]))}{x.size(0)} \tag{1}$$

3.3 Node Detection Network

The node detection network takes as input the convolutional feature vector generated by the backbone network and outputs a set of rectangular node proposals, each with an objectness score.

We slide a small convolutional network over the input layer. The output is fed into two sibling layers: a box-regression layer and a box-classification layer. At each sliding window, we predict one region proposal which encodes the four coordinates of a box and one score which estimates the probability of whether the proposal is a node or not. Each region proposal is parameterized relative to a reference box, called anchor. We parameterize the coordinates of the bounding boxes following Girshick et al. (2014):

$$t_x = (x - x_a)/w_a, t_y = (y - y_a)h_a,$$

$$t_w = log(w/w_a), t_h = log(h/h_a),$$

$$t_x^* = (x^* - x_a)/w_a, t_y^* = (y^* - y_a)/h_a,$$

$$t_w^* = log(w^*/w_a), t_h^* = log(h^*/h_a),$$

where x, y, w, and h is the box's center point and width and height. Variables x, x_a and x^* are for the predicted box, anchor box, and ground-truth box, respectively (likewise for y, w, h). This parameterization converts large integers of bounding box coordinates to variables close to interval $[-1, 1]$, and therefore improves the numerical performance of the algorithm.

To train the node detection network, we assign a binary class label of being a node or not to each anchor. We assign a positive number to two kinds of anchors: (1) The anchor with the highest Intersection-over-Union (IoU) with a ground-truth box, or (2) an anchor that has an IoU higher than 0.9 with any ground-truth box. We assign a negative number to an anchor if its IoU is less than 0.3 for all ground-truth boxes. An anchor that is neither positive nor negative does not contribute to the train objective. Because the number of positive labels and negative anchors may be different and therefore the train may bias toward one direction, we sample from the more to ensure the same size of positive and negative labels during training.

We apply binary cross entropy loss to the objectness, and smooth-l1 loss to the region proposal. The final loss is the sum of the objectness loss and region proposal loss over all anchors.

Table 3: Node recognition network architecture.

Layer Type	**Parameters**
Input	C × H × W
Basic Block	64, 3 × 3
Max Pool	3 × 3
Basic Block	128, 3 × 3
Max Pool	3 × 3
Basic Block	256, 3 × 3
Max Pool	3 × 3
Basic Block	LENVOC, 3 × 3

For prediction, we heuristically choose top 50 anchors with the highest scores, and group them by the condition that anchors with IoU greater than 0.2 are in the same group. Then, we choose from each group the highest score as the final prediction.

The node detection network is based on the network faster RCNN, Ren et al. (2015). Nevertheless, our network differs from the faster RCNN in two ways. (1) Because the nodes in flow charts have similar size and shape, we use one anchor at each sliding window, instead many anchors with different size and ratio, to save computing power. (2) Because the nodes in flow charts do not overlap, we consider proposals in the same group if one has IoU over a threshold with any other one in the group. Meanwhile, fast RCNN considers proposals in the same group if one has IoU over a threshold with the one of highest score.

The architecture of the node detection network is as follows. The intermediate layer is a Conv2D layer with a 3×3 kernel and 400 output channels. The classifier is a Conv2D layer with a 3×3 kernel and 1 output channel. The regressor is a Conv2D layer with a 3×3 kernel and 4 output channels. Each Conv2D layer is activated by ReLU function.

3.4 Node Recognition Network

The node recognition network takes as input the crop of each node and outputs a vector of size $LENVOC \times 1 \times W$, where $LENVOC$ is the vocabulary size. Table 3 is the architecture of the node recognition network, where the base block shares the same architecture as the basic block in the backbone network, Table 1,

The output of the node recognition network is interpreted as probability distribution over the vocabulary at each position along the width direction. CTC loss is a function for training sequential problems such as handwriting recognition and speech recognition. CTC loss does not attempt to learn the character boundaries, and can be applied if the input is a sequence with some order. By adoption CTC loss, we do not need to use another recurrent neural network to predict the text, considering that we already have four networks. Instead, we directly predict from the output of the node recognition network.

When prediction, we use greedy search to decode text from the output vector.

Optionally, a spatial transformer network (STN), Jaderberg et al. (2015), can be inserted before the node recognition network to further adjust the the boundary of the node crop.

3.5 Train and Implementation

The whole network is trained end-to-end. The lost function is the sum of loss of all sub-networks:

$$loss = loss_{edge} + loss_{ndc} + loss_{ndr} + \sum_{node} loss_{nr}, \tag{2}$$

where $loss_{edge}$, $loss_{ndc}$, $loss_{ndr}$ and $loss_{nr}$ is the loss of edge network, classifier and regressor of node detection network and node recognition network, respectively.

We use SGD method to train the network. Learning rate is 0.02, and halved when the error plateaued.

4 EXPERIMENT

We designed experiments to answer the following research questions: (1) what is the accuracy of FCR ? (2) what is the performance of FCR? (3) Is FCR able to synthesize real-world programs?

The experiments are conducted on a desktop with Geforce 1070 GPU, Intel i7 CPU, and 16GB memory. FCR is implemented in PyTorch pyt.

We tested the following networks.

- FCR: which is our basic network
- Separated FCR: not sharing the backbone network. See Section 4.4.
- FCR with ResNet: add optional residual learning. See Section 3.1.
- FCR with STN: add option STN network. See Section 3.4.
- FCR with ResNet and STN.

4.1 DATASET GENERATION

We created a synthetic dataset to train and test the FCR, since there are no existing datasets. A data sample includes a flow chart image in PNG format and a text file containing the ground truth data. The ground truth data includes an adjacent matrix for edge information, four coordinates for each bounding box and a character string for each node. The train and test dataset contains 9960 and 2490 data samples, respectively.

To generate a data sample, we first generate the adjacent matrix. Secondly, we generate the text for each node. Then, we draw the image using an popular open-source software, GraphViz gra . Finally, we use a heuristic algorithm to computer the coordinates of each bounding box in the image, and the computed bounding boxes are used as ground truth data.

We enumeratively generate all adjacent matrixes of 3 to 6 nodes, 0 to 2 of which are decision nodes. Note that a decision node has two branches: `YES` and `NO`. We paddle all matrixes to size 6×6 with zeroes.

We use an alphabet of 50 characters, including lower case English characters, digit characters, arithmetic operators, parenthesis et al. Note that the alphabet can be chosen freely without changing the essential network architecture. The text in each node contains 3 to 9 random characters in the alphabet. We insert an *id* at the beginning of the text to match the text with the *id*-th row of adjacent matrix.

When drawing the flow chart, we set the maximal width to be 200 pixels and maximal height to be 400 pixels. We randomize the width of lines, including nodes boundaries and edges, between 1 to 5 pixels. We randomize the font size of characters between 20 to 30. We randomize the RGB dimensions of the font color between 0 to 255. Because we use an heuristic algorithm to generate ground truth data for bounding boxes, and designing and implementing algorithms for every shape requires considerable engineering effort, we use rectangles for the shape of nodes when drawing flow charts. However, given ground truth data for bounding boxes, our algorithm can be trained the same way to handle other shapes.

4.2 ACCURACY

We measure the Edge Accuracy, Sequence Accuracy, Nodes accuracy and Graph Accuracy of FCR prediction. Edge accuracy is the percentage of images whose edges are correctly predicted. We say the edges are correctly predicted if the predicted adjacent matrix is exactly the same as the ground truth. Sequence accuracy is the percentage of nodes whose text content is correctly predicted among all nodes in all images. Sequence accuracy shows how well the node recognition network works. Nodes accuracy is the percentage of images whose nodes are all correctly predicted. Graph accuracy is the percentage of images whose edge and nodes are all correctly predicted.

The first row of Table 4 shows the accuracy of FCR and its subnets. The Edge, Sequence, Nodes and Graph accuracy is 94.1%, 90.6%, 67.9% and 66.4%, respectively. Note that the Graph accuracy is the joint probability of all edges and all nodes, so it is less than Edge and Sequence accuracy.

Table 4: Accuracy in percentage of FCR. The columns are Edge, Sequence, Nodes and Graph accuracy, respectively.

	Edge	**Squence**	**Nodes**	**Graph**
FCR	94.1	90.6	67.9	66.4
+STN	90.0	93.8	71.3	64.4
+ResNet	91.9	91.3	64.8	60.3
+STN+ResNet	90.9	93.2	70.0	63.2
Separated	93.7	91.1	68.2	65.8

Table 5: Time cost in milliseconds of FCR . The columns are the time cost of the Backbone, Edge, Node Detection, Node Recognition network and FCR, respectively.

	BB	**Edge**	**ND**	**NR**	**FCR**
FCR	3.5	0.4	42.4	14.2	60.5
+STN	3.5	0.4	43.3	16.6	63.8
+ResNet	3.5	0.4	44.8	14.0	79.1
+STN+ResNet	3.6	0.5	47.0	16.7	84.3
Separated	3.5	0.4	42.8	15.1	61.8

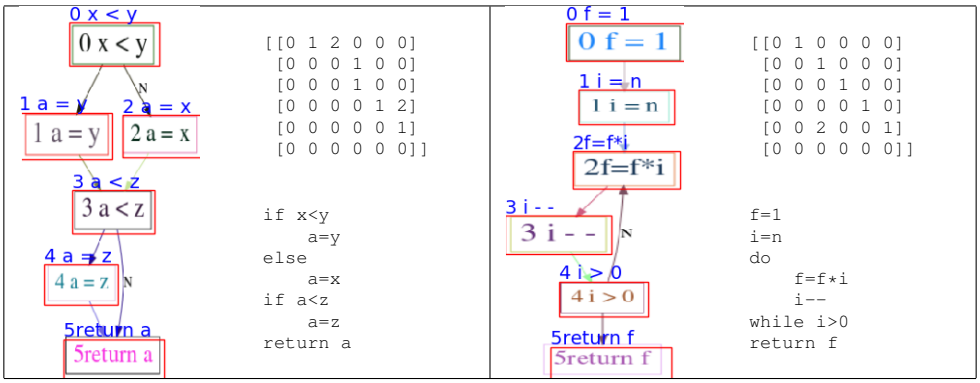


Figure 3: Samples of FCR prediction. The predicted bounding boxes are drawn in red lines. The predicted text is drawn in blue above the bounding box. The right figure is the predicted adjacent matrix and synthesized source code.

We conducted ablation study to sell how well the optional enhancements are. Rows 2-4 in Table 4 show the accuracy of FCR with different enhancements. We find that all STN enhancements improved the accuracy of node detection and recognition networks. We also find that the edge accuracy and graph accuracy decreased with those enhancements. Our explanation is that the extra trainable weights in the enhancements caused the network to bias toward the nodes detection and recognition networks, so caused the decrease in the Graph accuracy.

4.3 PERFORMANCE

Table 5 shows the performance of FCR and its subnets. The overall time cost of FCR is about 60 milliseconds, and the performance of other networks are close to the performance of FCR. Among the subnets, the time cost of node detection network is the major part, 70.1%, of the overall cost. The rest is the node recognition network (23.5%), backbone network (5.8%), and edge network (1%).

Table 6: Result of real-world program synthesis. The columns are program name, Graph accuracy, Edge accuracy, Nodes accuracy, number of nodes, correctly predicted nodes.

Program	Graph	Edge	Nodes	#Nodes	Nodes
abs	1	1	1	4	4
swap	1	1	1	3	3
max	0	1	0	4	3
sum	0	0	1	6	6
max3	1	1	1	6	6
log	1	1	1	4	4
radius	1	1	1	3	3
poly	1	1	1	6	6
factorial	1	1	1	6	6
quadrant	0	0	1	5	5
cntpos	0	0	0	6	3
sum	7	8	9	53	49
percentage	63.6	72.7	81.8	-	92.5

4.4 End-to-End vs Separated Network

Because edges and nodes information do not depend on each other, it is natural to consider to use two separate networks to predict edges and nodes. We designed experiments to compare the two different approaches.

We made two versions of FCR and modified as follows. For one version, we disabled the node detection network and node recognition network. For the other version, we disabled the edge detection network. We trained the two networks separately with the same dataset and hyper parameters.

The rows *Separated* in Table 4 and 5 show the accuracy and performance of the separated networks. The accuracy is close to FCR, therefore it is feasible to share the computation of the backbone network. The sharing saves the 3.5 milliseconds, which is 5.8% of the time cost.

4.5 Real-world Program Synthesis

In order to see whether FCR can synthesize real-world programs, we created a dataset of 11 programs, which are selected a programming textbook, Etter (1998). We manually wrote the programs and obtained their adjacent matrixes and node text. Then, we drew the images of the flow charts using GraphViz. We fed the images to FCR and then manually compare the output to the programs.

Table 6 is the testing result. It shows that 70% of programs are correctly predicted, 70% edges are correctly predicted, and 93.6% individual nodes are correctly predicted.

Figure 3 is two samples that visually demonstrate the input and output of FCR. The first one is the function that returns the max of three numbers. The second one is the `factorial` function.

5 Conclusion

We presented FCR, a deep convolutional neural network that simultaneously parses the edges and nodes information from the image of a flow chart. Experiments show that we can share the computation of the feature vector. FCR achieves 66.4%, 94.1%, 90.6% for graph, edge and node accuracy, respectively. FCR accheves close accuracy on a real-world dataset.

References

Graphviz. `https://www.graphviz.org`.

Multi-label margin loss. `https://pytorch.org/docs/stable/nn.html`.

Matej Balog, Alexander L Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. *arXiv preprint arXiv:1611.01989*, 2016.

Lin Cheng. Sqlsol: An accurate sql query synthesizer. In *International Conference on Formal Engineering Methods*, pp. 104–120. Springer, 2019.

Delores M. Etter. *Introduction to C*. Prentice Hall, 1998.

Yu Feng, Ruben Martins, Jacob Van Geffen, Isil Dillig, and Swarat Chaudhuri. Component-based synthesis of table consolidation and transformation tasks from examples. In *ACM SIGPLAN Notices*, volume 52, pp. 422–436. ACM, 2017.

Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 580–587, 2014.

Sumit Gulwani. Dimensions in program synthesis. In *Proceedings of the 12th international ACM SIGPLAN symposium on Principles and practice of declarative programming*, pp. 13–24. ACM, 2010.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Max Jaderberg, Karen Simonyan, Andrew Zisserman, et al. Spatial transformer networks. In *Advances in neural information processing systems*, pp. 2017–2025, 2015.

Tessa A Lau and Daniel S Weld. Programming by demonstration: An inductive learning formulation. In *International Conference on Intelligent User Interfaces: Proceedings of the 4 th international conference on Intelligent user interfaces*, volume 5, pp. 145–152. Citeseer, 1998.

Nicholas Locascio, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. Neural generation of regular expressions from natural language with minimal domain knowledge. *arXiv preprint arXiv:1608.03000*, 2016.

Zohar Manna and Richard Waldinger. A deductive approach to program synthesis. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 2(1):90–121, 1980.

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, pp. 91–99, 2015.

Gary B Shelly and Misty E Vermaat. *Discovering Computers, Complete: Your Interactive Guide to the Digital World*. Cengage Learning, 2011.

Armando Solar-Lezama and Rastislav Bodik. *Program synthesis by sketching*. Citeseer, 2008.

Chenglong Wang, Alvin Cheung, and Rastislav Bodik. Synthesizing highly expressive sql queries from input-output examples. In *ACM SIGPLAN Notices*, volume 52, pp. 452–466. ACM, 2017.

Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. Sqlizer: query synthesis from natural language. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):63, 2017.

Sergey Zherzdev and Alexey Gruzdev. Lprnet: License plate recognition via deep neural networks. *arXiv preprint arXiv:1806.10447*, 2018.